

Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 9: Data Processing: Import, Export, and Queues

Date-filtered reports, CSV export and import, and when heavy work belongs in the background

What You'll Learn

1. Build a report that aggregates data over a date range, and avoid a common pitfall when filtering by date
2. Export data to CSV without overloading server memory, and import CSV while handling bad rows without stopping the whole process
3. Recognize when a process should move to a **queue** instead of running directly inside the request cycle

This deck covers concepts. Applying reports, export/import, and queues to Simple POS happens in the practicum jobsheet.

Part 1

Reports and Data Aggregation

Four Basic Terms

ETL (Extract, Transform, Load): a common pattern of taking raw data, reshaping it, then storing or presenting it in a new form.

Batch Processing: processing many records at once in a group, rather than one at a time.

Queue: a mechanism for deferring heavy work so it runs in the background by a separate process, instead of directly during a user's request.

Import/Export: bringing data in from an external format (CSV, Excel) into the database, or sending it out to an external format to be read outside the application.

Reports Answer a Different Question

An ordinary list page answers "what exists": show every article, every transaction, every user. A report page answers a narrower but heavier question: "what's the total X over a given time range".

```
$articles = Article::whereBetween('published_at', [  
    $from, $to,  
])->get();  
  
$totalKata = $articles->sum('word_count');
```

`whereBetween` filters rows whose date falls between two bounds, then `sum()` adds up one column from that filtered result. The same pattern works for totaling anything already stored per row.

A Common Pitfall: The Cut-Off Last Day

A date filter for "August 1 through August 31" written literally means "through 00:00:00 on the 31st", which excludes every transaction that happens during the 31st itself.

```
$from = $request->date('from')->startOfDay();  
$to = $request->date('to')->endOfDay();
```

`startOfDay()` turns the start date into 00:00:00, and `endOfDay()` turns the end date into 23:59:59, so the whole last day counts, instead of being cut off at midnight.

Stored Aggregates, Not Recomputed

Meeting 6 established: a value like a transaction total can't be trusted when it arrives directly from client input, and must be recomputed on the server when the transaction is created. Once that value is correctly stored, a report may use it as-is.

```
// The report just sums a column that's already stored  
$totalPenjualan = $transactions->sum('total');
```

Recomputing a total from detail rows one by one, every single time a report is opened, burdens the server for no reason: the number has already been correct since the transaction was created.

Part 2

CSV Export and Import

Exporting Without Overloading Memory

The simplest way to export CSV is to collect every row into one large string, then send it all at once. That works for a few dozen rows, but once the row count reaches the tens of thousands, the entire file has to fit in server memory before anything is sent at all.

```
return response()->streamDownload(function () {  
    $out = fopen('php://output', 'w');  
    fputcsv($out, ['Title', 'Author', 'Date']);  
    foreach ($articles as $a) {  
        fputcsv($out, [$a->title, $a->author->name, $a->published_at]);  
    }  
    fclose($out);  
}, 'article-report.csv');
```

`streamDownload()` writes each row straight to the output stream while sending it to the browser, without holding the entire file in server memory first.

Importing: A Bad Row Shouldn't Stop Everything

Import runs the opposite direction: reading an uploaded CSV file, then turning each row into a new database row. The risky part isn't the reading and writing, it's handling a bad row without stopping the entire process because of it.

```
foreach ($rows as $row) {  
    $validator = Validator::make($row, [  
        'title' => ['required', 'string'],  
        'author_id' => ['required', 'exists:authors,id'],  
    ]);  
  
    if ($validator->fails()) {  
        continue; // skip this row, move to the next one  
    }  
  
    Article::create($validator->validated());  
}
```

Part 3

When to Use a Queue

Two Ways to Process Work

Directly (in the request)

- The user waits until it's done
- If it fails, the user finds out right away
- Suits short work, where the result is needed immediately

Through a Queue (a job)

- Instant response, work happens in the background
- The user isn't left waiting in the browser
- Suits heavy work or work with unpredictable duration

Importing a few dozen rows finishes in seconds, fine to run directly. Once the volume climbs into the thousands, the same process can take tens of seconds or more, and HTTP requests have a timeout.

Signs a Process Belongs in a Queue

- Its duration can't be predicted in advance, depending on the size of the data being processed
- The result doesn't need to be seen by the user instantly on the same page
- It involves calling a slow external service, such as sending an email or calling a third-party API

A report that's opened and read directly on the page is usually still better processed directly, because the user is genuinely waiting for the result right then. A queue isn't a solution for all heavy work, only for work that doesn't need an instant result.

The Job Flow in Laravel

Controller



dispatch() a job



jobs table



Worker (queue:work)

```
php artisan make:job ProcessArticleImport
```

```
ProcessArticleImport::dispatch($csvPath);  
  
return back()->with('success', 'Import is being processed in the background.');
```

The controller just saves the file and sends one job to the queue, then immediately responds to the user, without ever waiting for the process to finish. The actual work (the validate-and-save loop) moves into the job class's `handle()` method.

Worker: A Separate Process That Runs Jobs

Worker: a separate, continuously running PHP process that picks up jobs from the `jobs` table one at a time, then runs their `handle()` method.

```
php artisan queue:work
```

Without a running worker, jobs just pile up in the `jobs` table and never get processed, because dispatching only saves the job to the queue, it doesn't run it directly.

If `QUEUE_CONNECTION` in `.env` is set to `sync`, a job runs immediately in place and never actually enters a real queue, erasing the entire benefit of moving it to the background.

Applying This to Simple POS

You'll apply the report, CSV export/import, and queue concepts directly to Simple POS in the practicum jobsheet: building a sales report with a date filter, exporting that report to CSV, building a product import from CSV, then moving that import to a queued job so users don't wait in the browser.

Full code: github.com/se-polinema/simple-pos-ch08

Summary (1/2)

- A report aggregates data over a date range via `whereBetween` , with `startOfDay()` / `endOfDay()` so the last day isn't cut off at midnight
- A value that's already correctly stored (like a transaction total) can be used as-is in a report, no need to recompute it every time the report opens
- CSV export uses `streamDownload()` to write rows straight to the output without holding the entire file in memory

Summary (2/2)

- CSV import validates each row one at a time and skips a failing row via `continue` , instead of stopping the entire process
- A process with unpredictable duration or large volume should move to a queued job, so the user gets an instant response while the process runs in the background
- A dispatched job only enters the `jobs` table, and doesn't actually run until a worker (`queue:work`) processes it

References & Discussion

Official Laravel documentation (Queues, File Storage, HTTP Responses)

Full code: `github.com/se-polinema/simple-pos-ch08`

Next meeting: Designing and Building a REST API